Chapter 7 - User Interface Design

- **UI** refers to the graphical elements shown on screen, while **UX** focuses on the overall experience and feelings of the user.
- Good graphics **do not always guarantee** good user experience.
- Users judge systems by their **interface**, not just features.
- Bad UI can cause **catastrophic errors** (e.g., deleting important files).
- Many software systems fail simply because of poor interface design.
- That's why **user interface design** is a key part of software development.
- It should make sure that users can control and use the system easily and effectively.
- UI must match users' skills, experience, and expectations.

Human Factors in User Interface Design

1. **People have limited short-term memory** 短期记忆有限
   - People can remember only about **7 things at a time**.
   - **If a system shows too much at once, users may get confused or make mistakes.**

Example: A form asking for 15 pieces of info at once can overwhelm users.

2. **People make mistakes**
   - Everyone makes mistakes, and **bad system messages or loud alarms** can make things worse by causing stress.

Example: A confusing error message like "Error 402" doesn't help users fix the problem.

3. **People are different**
   - Everyone has different physical abilities.
   - **Design should work for all users, not just the designer.**

Example: A small button might be hard to click for someone with shaky hands.

4. **People prefer different ways to interact** 喜欢不同的交互方式
   -Some people like **images**, others prefer **text**.

Example: A tutorial with both pictures and step-by-step instructions works better for more users.

UI Design Process

- UI design is iterative (repeat until perfect) that requires **close collaboration between users and designers**.
- The three main activities are:
  - **User analysis** – Learn what users need and how they will use the system.
  - **System prototyping** – Create sample versions (prototypes) of the interface for testing.
  - **Interface evaluation** – Test these prototypes with users to see what works and what doesn't.
- Explanation:
  - **Iterative means "try, fail, improve."** Example: TikTok's UI was tested 100+ times before launch.
  - **User analysis**: Ask: "Who will use this? What do they need?" Example: A banking app needs security; a game needs fun.
  - **Prototyping**: Start with paper sketches, then digital mockups.
  - **Evaluation**: Watch real users try the prototype. Example: If they keep tapping the wrong button, move it.

UI Design Principles

- UI design must consider the needs, skills, and abilities of the users.
- Designers should remember that people have limits (like short-term memory) and that mistakes happen.
- There are general UI design principles to guide the process, but not every principle fits every design.

1. Learnability

- The software should be **easy to learn** for quick user engagement.
- UI should be easy to learn, especially for **first-time users**.
- High learnability means users understand functions, navigation, and purpose quickly without confusion.
- How fast they can learn, how fast they can understand the UI

➢ How to apply:

- Design a clear, simple UI with visible labels (e.g., use "Play" instead of an unfamiliar icon).
- Add short, in-app tutorials for new users (e.g., swipe demo like in TikTok).

★ Users can start using the app quickly without training or frustration.
★ Reduces bounce rate—users won't leave due to confusion.

Example:

Good: TikTok's UI is simple—swipe up for likes, left/right for next video. Users learn in seconds.

Bad: A tax software with no tutorials or labels (e.g., a button named "Fiscal Aggregation" instead of "Calculate Tax").

2. **User familiarity**

- Incorporate features and elements familiar to users (e.g., form layouts, icons).
- Using widely accepted symbols, terms, and layouts to increases acceptance and ease of use.

➢ How to apply:

- Use common icons (e.g., 'trash can' for delete, 'disk' icon for save).
- Stick to standard form layouts (e.g., Name, Email, Password fields stacked vertically).

★ Familiar elements make users feel comfortable and confident.
★ Reduces learning time because users already understand the design pattern.

Example:

Good: A "trash can" icon for delete (used in Windows, Mac, phones).

Bad: Using a "fire hydrant" icon for delete (users would be confused).

3. **Consistency**

- **Same** colors, fonts, button styles, and layouts **across all screens**.
- The look and feel must be consistent and similar

➢ How to apply:

- Use the same font, color, and button style throughout the app (e.g., all primary buttons in blue).
- Keep page structure and navigation menus in the same location on all screens.

★ A consistent design builds trust and makes navigation easier.
★ Users don't have to "relearn" controls on every screen.

Example:

Good: All "Submit" buttons in an app are green and at the bottom-right.

Bad: One "Submit" button is green, another is red, and a third is hidden under a menu.

4. Minimal Surprise

- Introducing new features or concepts should be done cautiously, **avoid unexpected changes** that reduce familiarity/confuse users.
- The user will not be will not be happy because they need to learn & understand again

➢ How to apply:

- Keep button positions and behaviors stable across updates (e.g., always place "Save" in the same spot).
- Introduce new features with tooltips or “What’s New” banners to avoid shocking users.

★ Predictable interfaces help users feel in control.

★ Sudden changes confuse users and cause errors.

Example:

Good: Gmail’s "Undo Send" appears in the same spot every time.

Bad: An app suddenly moves the "Save" button to a different corner without warning.

5. Recoverability

- The interface should provide mechanisms allowing users to **recover from errors.**
- This can include helpful features like:
  - An **undo** button to fix mistakes
  - **Confirming** actions that could cause data loss
  - Using **soft deletes**, where deleted items can be recovered later

➢ How to apply:

- Add "Undo" or "Restore" features for actions (e.g., undo delete in emails)
- Show a confirmation dialog before deleting important data (“Are you sure you want to delete?”).

★ Users feel safe knowing they can fix mistakes.

★ Prevents frustration or panic from accidental actions.

Example:

Good: Google Docs auto-saves versions. Users can revert to older drafts.

Bad: A banking app deletes a transaction with no "Undo" option.

6. User Guidance

- The interface should give **meaningful feedback** when errors occur and provide context-related help to users.
- Provide help when users need it—tooltips, error hints, tutorials.
- Some user guidance such as help systems, on-line manuals should be supplied

➢ How to apply:

- Show clear error messages (e.g., “Password must be 8+ characters, including a number”).
- Include tooltips, FAQs, or a Help button (e.g., info icons in form fields).

★ Users need support when facing errors or unfamiliar actions.
★ Guidance improves user satisfaction and success rate.

Example:
Good: A password field says: "Must include 1 number and 8+ characters."
Bad: A blank error message: "Invalid input. Try again."

7. User Diversity

- The interface should provide appropriate interaction mechanisms for diverse types of users and for users with different capabilities (blind, poor eyesight, deaf, colour-blind)
- Design for all users—different ages, cultures, disabilities.

➢ How to apply:
  - Use both icons and text (e.g., "Go 🟢") for color-blind users.
  - Provide adjustable font sizes or screen reader support for the visually impaired.

★ Inclusive design ensures everyone can use the software, regardless of ability.
★ Increases your app's reach by serving a wider audience.

Examples:
Color-blind users: Use symbols + colors (e.g., 🟢+ "Go" for traffic lights).
Elderly users: Allow font size adjustment (like iPhone's zoom feature).
Mental health: Avoid flashing bright red for anxiety-prone users (use softer colors).

UI Design Issues

- User interface design should solve two main questions:
  - How will the user **interact** with the software?
  - How will the software **show information** to the user?
- It must **combine** both interaction and presentation smoothly.
- Designers need to **find a balance** between the best way to interact, the best way to show information, the users' experience, and the devices being used.

Interaction: Exp: A voice-controlled thermostat vs. manual buttons.

Presentation: Exp: A weather app shows temps as numbers (digital) or a sun/cloud icon (analog).

User Interaction Styles

- User interaction means the user gives **commands** and **inputs data** to the software to make it do something.

- Each style fits different needs. Example:
  - Menu selection: Best for beginners (simple choices).
  - Command language: For experts (faster but harder to learn).

1. **Question-answer**

- The user asks the software a **single question**, and the software gives **one answer**.
- It's a simple, one-step exchange.

*Example: Typing a question into a **chatbot** and getting a reply. (e.g., "What's the weather?").*

2. **Direct manipulation**

- The user **controls objects** on the screen using a device like a **mouse or touch**.
- Actions are done by **clicking, dragging, or tapping** objects.

Example: Dragging a file to the trash bin or tapping icons on a phone.

3. **Menu selection**

- The user **chooses** a command from a list (menu).

Example: Clicking "Save As" from the File menu.

4. **Form fill-in**

- The user **enters** information into fields on a form.
- Forms may include dropdown menus and action buttons.

Example: Filling out an online sign-up form and clicking "Submit".

5. **Command language**

- The user **types** commands and any needed details (parameters) to control the software.

Example: Typing `copy file.txt folder/` in a command-line interface.

6. **Natural language**

- The user gives commands using **everyday language** (like English).
- The system **translates** this into actual software commands behind the scenes.

Example: Typing or saying "Show me today's schedule" and the app opens your calendar.

Each style fits different needs. Example:

- Menu selection: Best for beginners (simple choices).
- Command language: For experts (faster but harder to learn).

Information Presentation

- is about how the system shows information to the user.
- It can be shown directly, like text in a word processor.
- Or it can be transformed, like showing data as a graph or chart.

Example: A weather app showing the forecast as icons (sun, rain) instead of plain text.

- A good design **separates the data** from how it is **shown** to the user.
- MVC (Model-View-Controller): A coding method to keep data and display separate.
    - Model: Stores the data
    - View: Displays the data
    - Controller: Handles user input

Analog vs. Digital Presentation

a. Digital presentation
    - Compact – uses little screen space
    - Shows exact, precise values

Example: A digital clock showing 12:45 PM

b. Analog presentation
    - Gives a quick overall view of a value
    - Good for showing relative differences
    - Makes it easier to spot unusual or extreme values

Example: A speedometer or a progress bar

Data Visualization

- This is about **how to display large amounts of information** clearly.
- Visualization helps users see **patterns**, **connections**, and **trends** in data.

Examples of data visualizations include:

- Weather maps showing temperature and rain
- Phone system status (e.g., busy or idle)
- Chemical plant dashboards with pressure and temperature gauges
- 3D models of molecules
- Websites shown as a tree diagram to explore links easily

Colour Displays

- **Colour** can make an interface clearer and help users understand **complex information** & enhancing UI aesthetics.
- It can also be used to **highlight important or unusual events** (like errors).
- However, there are some **common mistakes** in using colour:
  - Over-reliance on color to convey meaning, which excludes colorblind users.
  - **Using too many colours**, which can make the screen look messy and confusing.

Colour Use Guidelines

➔ Limit main colors to 2-3 for visual coherence 视觉连贯性 (example: Apple uses mostly black, white, gray with occasional blue).

- ◆ *This helps create a clean and professional look, making the interface easier to navigate.* 简洁专业的外观

➔ Use color coding consistently to support user tasks, but customization can be allowed. 一致的颜色,但进行自定义

- ◆ *Consistency builds user familiarity and reduces confusion during interaction.*

➔ Design for monochrome 单色 (group of colors from same family) before adding accent colors 强调色.

- ◆ *Starting with a simple palette ensures readability and a balanced visual hierarchy.* 从简单的调色板开始，确保可读性和平衡的视觉层次

➔ Avoid color clashes 颜色冲突 (Contradict colors 矛盾色) that create visual discomfort by "fighting" for attention. 因争夺注意力而造成视觉不适

- ◆ *Reducing visual noise helps users focus on important content without distraction.*

➔ Use contrasting colors that complement each other instead. 互补的对比色

- ◆ *Complementary colors improve clarity and guide user attention effectively.* 提高清晰度并有效地引导用户注意力

➔ Use color changes to indicate software status (e.g., progress bars).

- ◆ *Dynamic color feedback provides users with immediate and intuitive system responses.* 动态颜色反馈为用户提供即时直观的系统响应

➔ Facilitate access for colorblind users by avoiding problematic color combinations (e.g., red/green). 避免使用不合适的颜色组合（例如红/绿），方便色盲用户使用

- ◆ *Inclusive color choices ensure that all users can understand the interface equally* 包容性的颜色选择可确保所有用户都能平等地理解界面

➔ Do not depend on color alone for conveying 传达 important information; use symbols, icons, and text alongside colors.

- ◆ *Multiple cues enhance comprehension and prevent misunderstandings across diverse user groups.*

User Guidance

- helps users with online help, error messages, and manuals.
- It should be part of the interface to assist users when needed or when errors happen.
- Help and messages should work together smoothly.

1. Help System Use

- should be context-sensitive, integrated with the UI, and offered at multiple entry points especially where errors are common.
- Users should be able **to get help** from different places.
- The help system should show where the user currently is.
- It must be easy to move around and **find information** within the help system.

2. Error Message

- very important & should clearly describe the error and its consequences.
- Bad messages can make users avoid the system.
- Be polite, concise, consistent, constructive, and suitable to the target user (e.g., formal for adults, friendly for kids).
- Provide instructions on how to recover or undo errors.
- Use a combination of text, icons, colors, audio (alarms), and visuals for effective communication.
- Avoid making users feel guilty or afraid after errors; promote learning and recovery.
- They should match the user's background and experience.

3. Help Information

- Documentation should not just be an online manual.
- Screens don't match well with paper pages.
- Dynamic displays can show information better.
- People read printed text more easily than screen text.

4. User Documentation

- should cater to a diverse audience with varying technical skills
- Systems should include both online help and paper documentation.
- Documentation must suit users of all skill levels, from beginners to experts.
- Besides manuals, simple guides like quick reference cards should also be provided.

Document Type 文档类型

1. Functional description 功能描述

→ A short summary of what the system does. 系统功能的简要概述

2. Introductory manual 入门手册

→ An easy, informal guide to get started with the system. 系统入门的简单、非正式指南

3. System reference manual 系统参考手册

→ A detailed guide explaining all system features. 详细解释所有系统功能的指南

4. System installation manual 系统安装手册

→ Instructions on how to install the system. 系统安装说明

5. System administrator's manual 系统管理员手册

→ Guidance on managing and maintaining the system during use. 系统使用过程中的管理和维护指南

User Interface Evaluation 用户界面评估

- User interface designs should be tested to see if they work well.
- Full evaluations are often too costly and hard to do. 全面评估通常成本过高且难以进行
- Ideally, interfaces are checked against usability standards, but these are rarely made.
- Measuring user-friendliness involves evaluating five usability attributes:

1. Learnability 易学性 → How quickly new users become productive 新用户上手的速度.
    - Tested by timing task completion with and without training, and recording error counts. 通过计时任务完成情况（包括培训和未培训）并记录错误次数进行测试。

2. Speed of Operation 操作速度→ How fast users can complete tasks compared to manual/previous methods. 与手动/之前的方法相比，用户完成任务的速度有多快

3. Robustness 稳健性 → System's tolerance to user errors without crashing or hanging. 系统对用户错误的容忍度，不会崩溃或挂起
    - Can be tested with automated tools simulating many errors. 可以使用模拟多种错误的自动化工具进行测试

4. Recoverability 可恢复性→ Effectiveness of error messages and user's ability to undo or recover from mistakes. 错误消息的有效性以及用户撤消或恢复错误的能力

5. Adaptability 适应性 → User acceptance and satisfaction with the UI, measured via surveys, interviews, or questionnaires. 用户对用户界面的接受度和满意度，通过调查、访谈或问卷调查进行衡量

Simple evaluation techniques 简单的评估技巧

- Adding code to track how features are used and record user errors.
  添加代码来追踪功能的使用情况并记录用户错误
- Using questionnaires to gather user opinions.
  使用问卷调查收集用户意见
- Recording users with video to watch and analyze later.
  用视频记录用户操作，以便日后观看和分析
- Providing a button for users to send feedback directly while using the system.
  提供按钮，方便用户在使用系统时直接发送反馈